

Kev on NVIDIA Jetson: TensorRT, ExecuTorch and Low-Precision Measurements

Measurement record for jaredpalmer/kev#71 (fork ZexinLi0w0/kev, branch jetson-tensorrt)

Zexin Li

2026-09-29

Abstract

We measure the Kev decision models (a Jev-style reconstruction on Qwen3.5 bases) on an RTX 6000 Ada workstation GPU, a Jetson AGX Orin 32 GB and a Jetson Orin Nano 8 GB, following Kev’s own README serving benchmark. Kev’s PyTorch path runs on the AGX but not on the Nano. We make the Qwen3.5 checkpoints compilable by TensorRT through a TensorRT-lowerable Gated DeltaNet recurrence, a static-shape program for JetPack 6’s TensorRT 10.3, and a raw-engine runtime for 8 GB boards. With these, Kev-0.8B runs on the Orin Nano for the first time (767 / 513 ms new / cached, 0 answer flips vs fp32). On the AGX, TensorRT matches Kev’s PyTorch path only in mixed fp16; INT8 weight-only and SmoothQuant build and keep 0 flips; INT4 loses accuracy and cannot be converted by the Jetson Torch-TensorRT; FP8 is refused by the hardware. ExecuTorch’s XNNPACK program runs on the AGX CPU with fp32-exact answers; the TensorRT-delegate .pte route is blocked on Jetson.

1 Setup

Device	Software	Notes
RTX 6000 Ada (sm_89), server3	torch 2.15 nightly (cu130), Torch-TensorRT 2.15, TensorRT 11.3	shared machine; card power only
Jetson AGX Orin 32 GB (sm_87)	JetPack R36.4.7, CUDA 12.6, TensorRT 10.3, torch 2.8 / 2.5	29 GB visible; INA3221 rails
Jetson Orin Nano 8 GB (sm_87)	JetPack R36.5.0, CUDA 12.6, TensorRT 10.3, torch 2.8	7.6 GB shared CPU/GPU DRAM

Models: Kev-0.8B (all devices), Kev-4B and Kev-9B (AGX, PyTorch path). Kev’s PyTorch path on the AGX uses torch 2.5 with flash-linear-attention (Triton 3.8); on the server it uses the reference DeltaNet kernels.

2 Method

All serving numbers use Kev’s own `scripts/serving_bench.py` procedure, imported rather than re-written; `scripts/trt_serving_bench.py` only swaps the model behind `kev.serve.Server`.

- **Latency:** model time per request as the server reports it (`latency_ms`), median of 20 after two warm-ups, for a new state and for the same state again (prefix-cache hit), on upstream’s four request shapes: 2 or 6 questions on a short ticket, 5 questions on a 370-token and on a 2,200-token state.
- **Throughput:** requests/s with p50/p99 at 1, 8, 32 and 64 concurrent clients (a first pass to meet batch shapes, then a timed pass) on the README traffic, decision-v7 development traffic and 2,200-token documents.
- **Accuracy:** every non-PyTorch program is checked against fp32 PyTorch on `evals/smoke-v1` development (30 records, 40 questions): maximum $|\Delta p|$ and argmax flips. For scale, Kev’s README reports its own bf16 serving path at up to ~ 0.03 from fp32.
- **Energy:** INA3221 rails sampled at 100 Hz (AGX `VIN_SYS_5V0`; Nano `VDD_IN`). Server energy is not reported (shared card, `nvidia-smi` sees card power only).

3 What had to be built

- **A TensorRT-lowerable recurrence** (`kev/trt_ops.py`). Upstream runs the DeltaNet recurrence as a CPU or Apple-GPU custom op. We use the chunked algorithm with a static chunk count, exact masked padding

(pads get $\beta=0$ and no decay), and a blocked unit-lower-triangular inverse. A power-series inverse is exact in exact arithmetic but produced NaNs and 8–9 flips on real activations, because Kev’s L2-normalised keys make its terms grow to $\sim 10^{17}$. Verified against the token-by-token definition to $\leq 9 \times 10^{-6}$, including adversarial inputs.

- **A static-shape program for TensorRT 10.3** (`kev/trt_static.py`). JetPack 6’s TensorRT cannot build the dynamic-shape graph (“Could not find any implementation for node”). A fixed-shape prefill engine plus bucketed score engines with explicit lengths build and stay exact. TF32 had to be disabled: it moved the recurrent state by 4×10^{-4} .
- **Mixed fp16**: fp16 GEMMs with the recurrence, norms and pointer head in fp32, as strongly typed engines. This halves engine memory.
- **A raw-engine runtime for 8 GB boards** (`kev/trt_runtime.py`). Torch-TensorRT’s loader holds the archive, the base64 engine text and the engine at once ($\sim 3\times$). The engines themselves need only 273 MB and 155 MB of activation memory. Raw engines are extracted on the AGX (same sm.87 and TensorRT 10.3.0) and deserialised directly on the Nano, sharing one activation buffer. Peak GPU memory on the Nano is 1.0 GB.

4 Accuracy

Table 1: Parity vs fp32 PyTorch on smoke-v1 (max $|\Delta p|$ / argmax flips).

Program	Device	max $ \Delta p $	flips
Dynamic, fp32, TensorRT 11.3	RTX 6000 Ada	1.0×10^{-6}	0 / 40
Static, fp32 (eager)	Orin AGX	9.2×10^{-7}	0 / 40
Static, mixed fp16, TensorRT 11.3	RTX 6000 Ada	4.1×10^{-3}	0 / 32
Static, mixed fp16, TensorRT 10.3, raw-engine runtime	Orin Nano	4.8×10^{-3}	0 / 32
ExecuTorch XNNPACK fp32 (CPU)	Orin AGX	3.0×10^{-6}	0 / 40

The 32-question rows exclude 8 questions whose rows exceed that program’s 512-token context.

5 Serving results

Table 2: Latency, kev README serving procedure: median model time per request (ms), **new state** / **same state again**. [†] state longer than the program admits.

Device	Model	Path	2 q, short	6 q, short	5 q, 370 tok	5 q, 2,200 tok
RTX 6000 Ada	Kev-0.8B	PyTorch bfl6	94.3 / 48.5	105.1 / 59.9	128.8 / 57.7	278.6 / 60.4
RTX 6000 Ada	Kev-0.8B	TensorRT 11.3 dyn. fp32	46.1 / 11.2	71.7 / 36.1	81.8 / 31.2	234.6 / 35.5
Orin AGX 32GB	Kev-0.8B	ExecuTorch XNNPACK fp32 (CPU)	882.7 / 659.6	3406.4 / 3179.1	4517.4 / 2816.7	n/a [†]
Orin AGX 32GB	Kev-0.8B	PyTorch bfl6	245.9 / 126.1	250.8 / 130.8	250.0 / 129.9	448.4 / 133.2
Orin AGX 32GB	Kev-0.8B	TensorRT 10.3.0 static P=2240 fp32	1342.2 / 444.0	1342.0 / 446.4	1341.4 / 446.2	1342.7 / 446.8
Orin AGX 32GB	Kev-0.8B	TensorRT 10.3.0 static P=384 fp32	561.9 / 392.6	562.0 / 393.8	562.1 / 393.6	n/a [†]
Orin AGX 32GB	Kev-0.8B	TensorRT 10.3.0 static P=384 mixed fp16	235.4 / 155.4	235.6 / 156.4	235.5 / 156.4	n/a [†]
Orin AGX 32GB	Kev-4B	PyTorch bfl6	315.9 / 162.1	421.2 / 266.8	453.0 / 233.8	1511.5 / 288.1
Orin AGX 32GB	Kev-9B	PyTorch bfl6 (LoRA unmerged)	472.3 / 240.3	724.4 / 492.8	811.7 / 411.6	2798.8 / 466.7
Orin Nano 8GB	Kev-0.8B	PyTorch bfl6	error	error	error	error
Orin Nano 8GB	Kev-0.8B	TensorRT 10.3.0 static P=384 mixed fp16	766.2 / 513.3	767.0 / 514.7	767.2 / 514.8	n/a [†]

Table 3: Throughput (requests/s) at 1 / 8 / 32 / 64 concurrent clients, and p50 / p99 latency (ms) at 64 clients. Rejected = requests with a question row longer than the program’s largest engine.

Device	Model	Path	Traffic	req/s @1/8/32/64	p50/p99 @64	rej.
RTX 6000 Ada	Kev-0.8B	PyTorch bf16	5 questions, 2,200-token state	3.57 / 2.98 / 1.42 / 1.40	45646.9 / 45783.5	0
RTX 6000 Ada	Kev-0.8B	PyTorch bf16	6 questions, new short state	9.03 / 10.56 / 13.05 / 13.27	4807.8 / 4884.8	0
RTX 6000 Ada	Kev-0.8B	PyTorch bf16	decision-v7 development	9.52 / 10.51 / 14.63 / 14.72	4149.1 / 7545.2	0
RTX 6000 Ada	Kev-0.8B	TensorRT 11.3 dyn. fp32	5 questions, 2,200-token state	3.76 / 3.77 / 3.75 / 3.80	16790.4 / 16855.6	0
RTX 6000 Ada	Kev-0.8B	TensorRT 11.3 dyn. fp32	6 questions, new short state	12.98 / 13.27 / 13.14 / 13.09	4879.2 / 4944.9	0
RTX 6000 Ada	Kev-0.8B	TensorRT 11.3 dyn. fp32	decision-v7 development	17.52 / 17.84 / 17.91 / 17.92	3550.7 / 3589.7	24
Orin AGX 32GB	Kev-0.8B	PyTorch bf16	5 questions, 2,200-token state	2.20 / 1.30 / 0.65 / 0.62	102527.8 / 102613.6	0
Orin AGX 32GB	Kev-0.8B	PyTorch bf16	6 questions, new short state	3.91 / 4.78 / 6.48 / 6.95	9203.1 / 9240.2	0
Orin AGX 32GB	Kev-0.8B	PyTorch bf16	decision-v7 development	4.02 / 4.99 / 6.88 / 7.27	8625.2 / 9067.1	0
Orin AGX 32GB	Kev-0.8B	TensorRT 10.3.0 static P=384 fp32	5 questions, 2,200-token state	n/a		
Orin AGX 32GB	Kev-0.8B	TensorRT 10.3.0 static P=384 fp32	6 questions, new short state	1.77 / 1.78 / 1.77 / 1.78	36039.6 / 36115.4	0
Orin AGX 32GB	Kev-0.8B	TensorRT 10.3.0 static P=384 fp32	decision-v7 development	1.23 / 1.23 / 1.23 / 1.23	49267.1 / 57514.3	0
Orin AGX 32GB	Kev-0.8B	TensorRT 10.3.0 static P=384 mixed fp16	5 questions, 2,200-token state	n/a		
Orin AGX 32GB	Kev-0.8B	TensorRT 10.3.0 static P=384 mixed fp16	6 questions, new short state	4.18 / 4.22 / 4.22 / 4.21	15203.3 / 15265.5	0
Orin AGX 32GB	Kev-0.8B	TensorRT 10.3.0 static P=384 mixed fp16	decision-v7 development	3.06 / 3.07 / 3.07 / 3.06	19893.9 / 22811.8	0
Orin AGX 32GB	Kev-4B	PyTorch bf16	5 questions, 2,200-token state	0.66 / 0.29 / 0.17 / 0.17	387471.3 / 387562.8	0
Orin AGX 32GB	Kev-4B	PyTorch bf16	6 questions, new short state	2.36 / 2.44 / 2.53 / 2.54	25140.6 / 25183.4	0
Orin AGX 32GB	Kev-4B	PyTorch bf16	decision-v7 development	3.00 / 3.62 / 4.69 / 4.89	12784.9 / 14449.6	0
Orin AGX 32GB	Kev-9B	PyTorch bf16 (LoRA unmerged)	5 questions, 2,200-token state	0.35 / 0.16 / 0.09 / 0.09	717079.4 / 717171.4	0
Orin AGX 32GB	Kev-9B	PyTorch bf16 (LoRA unmerged)	6 questions, new short state	1.37 / 1.37 / 1.37 / 1.35	47080.2 / 48157.2	0
Orin AGX 32GB	Kev-9B	PyTorch bf16 (LoRA unmerged)	decision-v7 development	1.88 / 2.23 / 2.77 / 2.94	21166.4 / 25957.6	0
Orin Nano 8GB	Kev-0.8B	PyTorch bf16	5 questions, 2,200-token state	error		
Orin Nano 8GB	Kev-0.8B	PyTorch bf16	6 questions, new short state	error		
Orin Nano 8GB	Kev-0.8B	PyTorch bf16	decision-v7 development	error		
Orin Nano 8GB	Kev-0.8B	TensorRT 10.3.0 static P=384 mixed fp16	5 questions, 2,200-token state	n/a		
Orin Nano 8GB	Kev-0.8B	TensorRT 10.3.0 static P=384 mixed fp16	6 questions, new short state	1.29 / 1.30 / 1.30 / 1.30	49318.1 / 49347.1	0
Orin Nano 8GB	Kev-0.8B	TensorRT 10.3.0 static P=384 mixed fp16	decision-v7 development	1.30 / 1.30 / 1.31 / 1.30	49104.6 / 62882.0	24

Table 4: Energy per request (J) on the Jetson power rails (Orin AGX: VIN_SYS_5V0; Orin Nano: VDD_IN, whole module). Compare within a board only.

Device	Model	Path	2 q	6 q	370 tok	2,200 tok
Orin AGX 32GB	Kev-0.8B	ExecuTorch XNNPACK fp32 (CPU)	4.74	20.02	22.95	–
Orin AGX 32GB	Kev-0.8B	PyTorch bf16	1.34	1.56	1.66	3.01
Orin AGX 32GB	Kev-0.8B	TensorRT 10.3.0 static P=2240 fp32	8.22	8.10	8.30	8.35
Orin AGX 32GB	Kev-0.8B	TensorRT 10.3.0 static P=384 fp32	4.21	4.23	4.32	–
Orin AGX 32GB	Kev-0.8B	TensorRT 10.3.0 static P=384 mixed fp16	1.92	1.97	1.99	–
Orin AGX 32GB	Kev-4B	PyTorch bf16	2.18	3.70	3.87	10.52
Orin AGX 32GB	Kev-9B	PyTorch bf16 (LoRA unmerged)	3.50	6.59	7.08	20.20
Orin Nano 8GB	Kev-0.8B	TensorRT 10.3.0 static P=384 mixed fp16	8.16	8.12	8.21	–

Findings.

1. **The current Kev family now runs on an 8 GB Orin Nano**, only under TensorRT: 767 / 513 ms, 1.30 requests/s, ~ 8.2 J per request on the module rail, 0 flips. Kev’s PyTorch path does not run there. JetPack 6’s cuSOLVER cannot back the reference DeltaNet’s triangular solve, and flash-linear-attention’s Triton kernels do not load against that board’s torch.
2. **On the workstation GPU**, the fp32 TensorRT program beats Kev’s PyTorch bf16 path (46 vs 94 ms new, 11 vs 49 ms cached, short case). That baseline uses reference DeltaNet kernels; upstream’s README figures use flash-linear-attention and CUDA graphs.
3. **On the AGX, TensorRT only matches PyTorch, and only in mixed fp16**: 235 vs 246 ms new, 156 vs 126 ms cached. fp32 static engines are $2.4\times$ slower. Static shapes pay for padding, so every request costs the same.
4. **The TensorRT programs do not batch**: Kev’s server batches queued requests, while the static programs answer one at a time. Throughput is flat from 1 to 64 clients (AGX: 4.2 vs 7.0 requests/s for PyTorch at 64).

6 Low-precision quantization

Table 5: Low-precision quantization (NVIDIA ModelOpt 0.47) of the static mixed-fp16 Kev-0.8B program: backbone Linear layers only. Parity vs fp32 PyTorch on smoke-v1 (max $|\Delta p|$ / argmax flips, 32 questions).

Config	Device	Fake-quant	TensorRT build	Engine parity	p50 (ms)	Peak GPU (GiB)
INT8 weight-only	Orin AGX 32GB (sm.87)	0.012 / 0	built	0.013 / 0	234.486	2.01
INT8 SmoothQuant (W8A8)	Orin AGX 32GB (sm.87)	0.071 / 0	built	0.059 / 0	224.245	2.96
INT4 AWQ	Orin AGX 32GB (sm.87)	0.329 / 1	failed: no 4-bit converter	—	—	—
FP8	Orin AGX 32GB (sm.87)	0.050 / 0	failed: no FP8 hardware	—	—	—
INT4 blockwise W-only	Orin AGX 32GB (sm.87)	0.229 / 2	failed: no 4-bit converter	—	—	—
FP8	RTX 6000 Ada (sm.89)	0.037 / 0	built	NaN / 19	22.375	1.96
INT8 weight-only	RTX 6000 Ada (sm.89)	0.012 / 0	failed: env (ModelOpt ext.)	—	—	—

- **INT8 works on Orin.** Weight-only stays well inside the ~ 0.03 Kev accepts for bf16 serving; SmoothQuant is looser (0.059). Neither changes an answer. Neither is faster than mixed fp16 at this request size; weight-only lowers peak GPU memory (2.01 vs 2.90 GiB).
- **INT4 fails on two counts**: it already changes answers fake-quantized (1–2 of 32), and Torch-TensorRT 2.8, the only Jetson build, cannot convert 4-bit Q/DQ.
- **FP8 on Orin**: TensorRT refuses the build, “Networks with FP8 Q/DQ layers require hardware with FP8 support”. Orin’s Ampere-class tensor cores (sm.87) do FP16/BF16/TF32/INT8 but have no FP8 instructions, which need Ada (sm.89) or Hopper (sm.90). FP8 storage with dequantisation would still save memory, but gives no speed-up. On the RTX 6000 Ada the FP8 engines build and run (22 ms), but ModelOpt’s default recipe yields NaNs on this model.

7 ExecuTorch

XNNPACK on the AGX CPU: exported on the board with ExecuTorch’s own `examples/kev/export.py`, from a Python 3.13 environment because ExecuTorch’s aarch64 wheels are cp313/cp314 and JetPack 6 is 3.10. Kev#69’s two XNNPACK partitioner files from ExecuTorch main were applied over the wheel. The program is 3.08 GB and was exported in 11 min 54 s. Served through #69’s `ExecuTorchDecisionModel`: fp32-exact (3.0×10^{-6} , 0 flips), 0.9–4.5 s per request (Table 2).

TensorRT-delegate .pte on Jetson is blocked by three independent gaps: (1) no Jetson build of the exporter: `torch_tensorrt.executorch` needs Torch-TensorRT ≥ 2.15 , and JetPack 6 has 2.8; (2) a .pte’s engines load only on the TensorRT version and GPU that built them, so a server-built one cannot run on the boards; (3) no Jetson build of the delegate runtime (`torch_tensorrt_executorch_runtime`). On x86 the program was made to export with the ranges #69’s backend actually sends (one-token prefill, one-question score), and `load_program` now registers the TensorRT delegate. The export was still running at the time of writing.

8 Deviations and caveats

- **No throughput for the XNNPACK program.** Upstream's 1/8/32/64-client sweep would have taken ~ 4 h on the CPU. It was stopped at the user's request, restarted at 1 client, and that run was stopped too when all AGX workloads were cleared. Its latency is the full procedure.
- The AGX 2,240-token fp32 program's throughput was run on 2,200-token documents only; short traffic on it would only measure padding.
- The fixed-shape programs reject questions longer than their largest engine: 24 of 256 decision-v7 requests on the Nano and on the server's 128-row program. They are counted, not dropped silently.
- Kev-9B on the AGX was loaded with the LoRA unmerged: the fp32 merge needs ~ 36 GB.
- Energy rails differ between boards (AGX VIN_SYS_5V0, Nano VDD_IN), so compare energy within a board only.

9 Open items

x86 TensorRT .pte through #69's backend (parity and README benchmark); a 2,200-token program on the Nano; request batching in the static programs; Kev-4B under TensorRT (mixed fp16 / INT8); full decision-v7 parity for each program; ExecuTorch on the Orin Nano.

All raw JSON, cleaned logs and the scripts that produced every number are in `runs/trt-jetson/`, `scripts/` and `docs/jetson-tensorrt.md` of the fork.